IDENTITY AND ACCESS MANAGEMENT (IAM) IN AWS

DIFFERENCE BETWEEN THE SECRET KEY & ACCESS KEY

SECRET KEY

1. It is a private password used for security.
2. It is used to sign requests (like proving “this is me”).
3. It must be kept secret, never shared with anyone.
4. It works together with the Access Key.

ACCESS KEY

1. It is like a username or ID.
2. It is used to identify who is making the request.
3. It is safe to share in systems (but not publicly).
4. It works with the Secret Key for authentication.

Note:

1. Secret_key: Password
2. Access_key: Username

Both are required to log in and access AWS services.

FEDERATED USER

1. A federated user is someone who logs into AWS using another account (not AWS username/password).
2. AWS trusts another system (like Google or your company login) to verify the user.
3. The user does NOT exist as an IAM user in AWS.
4. They get temporary access (limited time login).

IDEA TO UNDERSTAND

1. Normal IAM User = Create account inside AWS
2. Federated User = Use existing login (Google/company)

EXAMPLE

1. You click on Login with Google
2. Google simply verifies you
3. AWS trusts Google account →so u get the aws account access

---

IAM TERMS

IAM terms define the process AWS uses to check identity and permissions before giving access.

1. **Principal** → Who is making the request (user/role/service)
2. **Request** → What the user is trying to do
3. **Authentication** → Verify identity (Are you real?)
4. **Authorization** → Check permission (Are you allowed?)
5. **Action** → What operation (like S3:PutObject)
6. **Resource** → On which service/resource (like S3 bucket)

EXAMPLE

User uploads file to S3

- Principal = User
- Action = Upload (PutObject)
- Resource = S3 Bucket
- Authentication = Login check
- Authorization = Policy check

SIMPLE TRICK TO UNDERSTAND

Imagine a security guard at a company gate:

1. Principal → You (person entering the office)
2. Request → "I want to enter the office"
3. Authentication → Guard checks your ID card (Who are you?)
4. Authorization → Guard checks permission list (Allowed or not?)
5. Action → What you want to do (enter, open room, etc.)
6. Resource → Office/room you want to access

IAM FEATURES

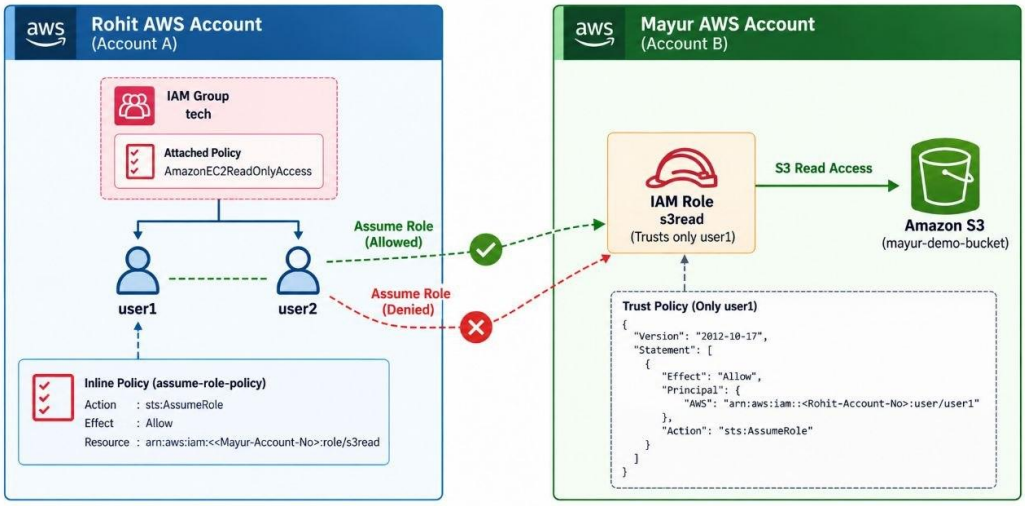
1. shared access to your aws account
2. Granular permissions
3. multifactor authentication
4. identity federation

5. secure access to aws resources

MULTIFACTOR AUTHENTIFICATION

1. Multi-Factor Authentication (MFA) in AWS is a security method where a user must provide two or more types of verification to log in.
2. So, even if someone steals your password, they cannot log in without the second factor (OTP/device).

PRACTICAL – CROSS ACCOUNT ACCESS USING IAM ROLE



1. Login with Rohit Account

1. create a group (tech)
2. create two users (user1, user2) and add in group (tech)
3. attach ec2-readonly access policy to a group

2. Login with Mayur Account

1. create a s3 bucket (mayur-demo-bucket)
2. create a role
    1. another aws account -----> (account no of Rohit)
    2. attach policy -----> S3ReadOnlyAccess -----> name (s3read)

3. Login Back to Rohit Account

1. go to IAM
2. click on group
3. add permissions
4. create Inlined policy
5. select a service ----> STS

6. Actions Allowed ---> AssumeRole
7. Effect ----------> allow
8. Resources ----> specific
9. Add ARNs
    1. Resource -----> Other Account (mayur-account-no)
    2. Role_name --> s3read
    3. Add ARNs
    4. Policy-Name ----> assume-role-policy
    5. Create policy

4.Login with user1

1. click on username
2. switch role---------> (mayur account no) --------> s3read-------> switch role
3. you can access the s3 bucket of mayur account

5. Login as user2

1. click on username
2. switch role---------> (mayur account no) --------> s3read-------> switch role
3. you can access the s3 bucket of mayur account

RESTRICT ACCESS ONLY TO USER1

6. Login as Mayur Account

1. click on created role(s3read)
2. click on trusted relationship
    1. edit trusted relationship
    2. paste ARN of user1 (arn:aws:iam::<Rohit-Account-No>::user/user1)
    3. update policy

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Principal": {
            "AWS": "arn:aws:iam::<Rohit-Account-No>:user/user1"
         },
         "Action": "sts:AssumeRole",
         "Condition": {}
      }
```

```
    ]
}
```

7.Login as user1

1. click on username
2. switch role ---------> (mayur account no) --------> s3read -------> switch role
3. you can access the s3 bucket of Mayur account

8. Login as user2

1. click on username
2. switch role ---------> (mayur account no) --------> s3read -------> switch role
3. you get a api error because user2 arn not be included in mayur account role.

AWS IAM BEST PRACTICES

1. avoid using root account except of account setup.
2. add user to group and assign permission to group.
3. use password policy or MFA
4. use access keys for CLI & SGK
5. never share ACCESS KEYS or password on any platform.
6. audit the permission using IAM credential report.
